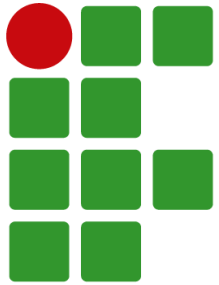

INSTITUTO FEDERAL
Goiano

Campus Trindade

Programação Web

Manipulação de dados

Node.js · Knex.js · PostgreSQL

Prof. Dr. Renato de Sousa Gomide

Objetivos da aula

- Relacionar **persistência** em banco com código **Node.js**
- Configurar o **Knex.js** como *query builder* para **PostgreSQL**
- Reconhecer o padrão **CRUD** em módulos de **modelo**
- Ler e escrever consultas com `.select`, `.where` (um ou **vários** critérios), `.insert`, `.join`
- Saber **encerrar o pool** de conexões com `db.destroy()` em scripts

Por que um banco relacional?

- Dados **estruturados** em **tabelas** com **tipos** e **restrições**
- **Integridade** por chaves primárias e estrangeiras
- Consultas com **SQL** (o Knex **gera** SQL a partir da API em JS)

> Neste projeto o domínio é uma **escola**: alunos, endereços, cursos e matrículas.

Stack

Runtime Node.js

Linguagem JavaScript

PostgreSQL

Modelo lógico (escola)

Entidade	Papel
aluno	Cadastro do estudante
endereco	Vários endereços por aluno (**1:N**)
curso	Oferta de curso
matricula	Aluno inscrito em curso (**N:N**), com data

Chave composta em **matricula**: `(id_aluno, id_curso)`.

O que é o Knex.js?

- **Query builder**: encadeamento de métodos → SQL **parametrizado**
- Menos concatenação manual de strings → ajuda a evitar **SQL injection**
- Mesma API útil para evoluir para **migrações** em projetos maiores

Não é obrigatoriamente um **ORM** completo (objetos ricos com lazy loading); aqui usamos **tabelas** e **objetos simples** (`insert` / `select`).

Configuração da conexão

Arquivo `src/db/index.js`: uma instância **`knex(config)`** exportada para todos os modelos.

```
const knex = require('knex');

const config = {
  client: 'pg',
  connection: {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
  },
  pool: { min: 2, max: 10 },
};

const db = knex(config);
module.exports = db;
```

CRUD em uma frase cada

Letra	Significado	Exemplo Knex
C	Create	`.insert(dados).returning('*')`
R	Read	`.select('*')` / `.where(...).first()`
U	Update	`.where({ id }).update(patch).returning('*')`
D	Delete	`.where({ id }).del()`

No **PostgreSQL**, `.returning(...)` traz a linha afetada logo após `insert` ou `update`.

Leitura — listar e buscar por id

```
const db = require('../db');

async function findAll() {
  return db('aluno').select('*');
}

async function findById(id) {
  return db('aluno').where('id', id).first();
}
```

Equivale mentalmente a `SELECT * FROM aluno` e `SELECT * FROM aluno WHERE id = ? LIMIT 1`.

WHERE com vários critérios (`AND`)

Em `src/model/endereco.js`, **dois parâmetros** restringem a busca (rua **e** número):

```
async function findByRuaAndNumero(rua, numero) {
  return db('endereco').where({ rua, numero }).select('*');
}
```

O objeto em `.where({ rua, numero })` vira `WHERE rua = ? AND numero = ?` com valores **parametrizados**. Forma equivalente: `.where('rua', rua).where('numero', numero)`.

Criação e atualização

```
async function create(aluno) {
  const rows = await db('aluno').insert(aluno).returning('*');
  return rows[0];
}

async function update(id, aluno) {
  const rows = await db('aluno').where({ id }).update(aluno).returning('*');
  return rows[0];
}
```

`returning('*')` evita outra ida ao banco só para montar o objeto retornado.

Remoção

```
async function remove(id) {
  return db('aluno').where({ id }).del();
}
```

Atenção a **chaves estrangeiras**: pode ser preciso apagar **matrículas** e **endereços** antes do aluno, conforme as `ON DELETE` do seu esquema.

Dados relacionados — duas consultas

Padrão: buscar a entidade principal, depois os filhos (claro e fácil de depurar).

```
async function findWithEnderecos(id) {
  const aluno = await findById(id);
  if (!aluno) return null;

  const enderecos = await db('endereco').where('id_aluno', id);
  return { ...aluno, enderecos };
}
```

Como em `src/model/aluno.js`: reutiliza `findById` em vez de repetir a consulta ao aluno.

Dados relacionados — `join`

Um curso com alunos matriculados (trecho simplificado do projeto):


```
async function findWithAlunos(id) {
```

Uso dos modelos no `index.js`

```
const { Aluno, Curso, Endereco, Matricula } = require('./model');

const novoAluno = await Aluno.create({
  nome: 'Roberto Gomide',
  email: 'roberto@email.com',
  data_nascimento: '1990-01-15',
});

const alunoComEndereco = await Aluno.findWithEnderecos(novoAluno.id);
```

Organização: `src/model/` exporta funções; `src/index.js` orquestra o fluxo de demonstração.

Encerrar o pool ao fim do script

```
async function main() {
  try {
    // ... chamadas aos modelos ...
  } catch (error) {
    console.error(error);
  } finally {
    const db = require('./db');
    await db.destroy();
  }
}

main();
```

`destroy()` libera conexões e permite que o processo **termine** sem ficar pendurado no *event loop*.

Boas práticas (resumo)

- **Um** módulo de configuração do Knex (`src/db/index.js`)
- **Modelos** por tabela ou agregado estável
- **Tratamento de erros** em operações assíncronas
- **Validação** de entrada antes de persistir (próximo passo didático)
- **Ordem consciente** em exclusões respeitando FKs

Próximos passos sugeridos

- Paginação (`limit` / `offset`)
- Ordenação (`orderBy`)
- Busca textual (`whereILike` no PostgreSQL)
- **API REST** com Express — roteiros no README.md (exercícios de implementação)
- **Migrações Knex** em vez de só `script.sql`

Referências

- Knex.js — documentação
- PostgreSQL
- Código e instruções detalhadas: README.md